

Informe Técnico

1. Portada

Trabajo Final: Compilador Subconjunto C++ en Java con ANTLR4

Integrantes: Maximo Bandini, Martino Ceconello

Materia: Compilación y Lenguajes de Programación

Profesor: Francisco Ameri

Fecha: Junio 2025

2. Introducción

Este informe describe el desarrollo de un compilador para un subconjunto de C++ usando ANTLR4 y Java 11. El objetivo fue implementar todas las fases de compilación (léxica, sintáctica, semántica, generación de código intermedio) y añadir una fase de optimización de instrucciones de tres direcciones, así como generar artefactos resultantes y un manual de usuario.

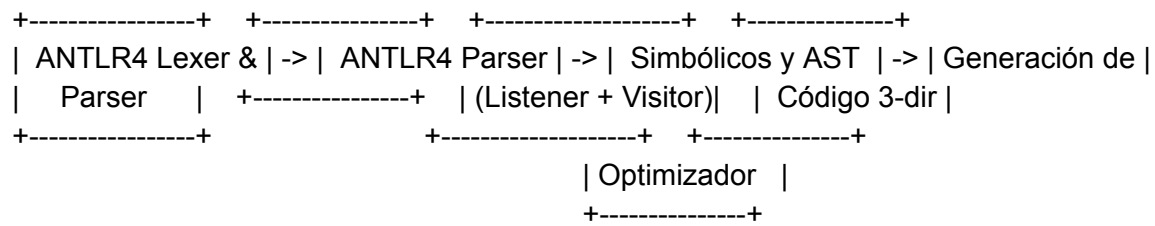
3. Análisis del Problema

Subconjunto de C++ implementado:

- Tipos de datos: `int`, `char`, `double`, `float`, `string`, `void`.
- Literales: enteros, decimales, cadenas, caracteres, URL, email, DNI, CUIL, fechas, teléfonos, código postal, IP, etc.
- Operadores aritméticos (`+` `-` `*` `/` `%`), comparadores (`<` `<=` `>` `>=` `==` `!=`), lógicos (`&&` `||` `!`).
- Control de flujo: `if-else`, `while`, `for`, `break`, `return`.
- Declaración y llamada de funciones con parámetros.

4. Diseño de la Solución

4.1 Arquitectura General



4.2 Fases de Compilación

1. **Léxico:** ANTLR4 separa tokens y captura errores léxicos.
2. **Sintaxis:** Construcción del AST y detección de errores sintácticos.
3. **Semántica:** `SimbolosListener` gestiona ámbitos, tabla de símbolos, chequeo de declaraciones y usos.
4. **Generación de Código Intermedio:** `CodigoVisitor` produce instrucciones de tres direcciones usando `GeneradorCodigo`.
5. **Optimización:** Fase que implementa:
 - Eliminación de código muerto.
 - Propagación de constantes.
 - Simplificación de expresiones constantes.
 - Eliminación de sentencias redundantes.
6. **Salidas:** Escritura de archivos de código intermedio y optimizado, resumen de compilación.

4.3 Decisiones de Diseño

- Uso combinado de Listener y Visitor para separar responsabilidades.
- Código intermedio en tres direcciones por claridad y facilidad de optimización.
- Implementación de optimizaciones locales sin reconstruir el AST.
- Directorio de salida determinado por `System.getProperty("user.dir")`.

5. Implementación

5.1 Gramática ANTLR4

- Archivo `MiLenguaje.g4` define reglas de parser y tokens, incluyendo fragmentos para dígitos, letras y símbolos.

5.2 Tabla de Símbolos

- Clase `TablaSimbolos` con lista de `Simbolo` (nombre, tipo, categoría, ámbito, línea, flags).
- Métodos para agregar, buscar y listar símbolos.

5.3 Generación de Código

- `CodigoVisitor` recorre el AST, invocando a `GeneradorCodigo` para producir instrucciones:
 - `t = a + b, goto, if !cond goto L, param, call, return.`

5.4 Optimizador

- Implementa cuatro métodos:
 1. `eliminarCodigoMuerto()` marca y filtra instrucciones inalcanzables.
 2. `propagarConstantes()` detecta y sustituye asignaciones constantes.
 3. `simplificarExpresiones()` evalúa operaciones constantes.
 4. `eliminarSentenciasRedundantes()` quita asignaciones `x = x`.

5.5 Clase Principal `App.java`

- Coordina todas las fases, captura errores y warnings.
- Imprime estadísticas y escribe archivos de salida en el directorio de ejecución.

6. Ejemplos y Pruebas

6.1 Casos de Prueba

1. Funciones con parámetros.
2. `if-else, while, for`.
3. Errores semánticos (variable no declarada, reasignación de `const`).
4. Optimización de bucles inalcanzables y expresiones constantes.

6.2 Resultados

- Generación de `ejemplo_codigo_intermedio.txt` y `ejemplo_codigo_optimizado.txt`.
- Reducción de instrucciones muertas y temporal innecesarios.

7. Dificultades y Soluciones

- Adaptación de switch expressions a Java 11.
- Manejo de rutas relativas y absolutas en PowerShell.
- Manejo de terminales Windows y caracteres especiales.

8. Conclusiones

Se logró un compilador completo para un subconjunto de C++, con análisis, generación de código y optimización, aplicando técnicas de diseño de compiladores y ANTLR4.